Matthew McKay, Ph.D.

Tessaly Health synthetic data generator

A seeded Python generator that builds three years of workforce, staffing and learning data for **Tessaly Health**, a fictional nonprofit regional health system of about 4,000 staff. It's the shared data source for every case study and data-based library item in this portfolio.

Method demonstration, not findings. Every row is synthetic. Turnover, vacancy, time to recruit, wages, benefits, agency bill rates and turnover cost are tuned to national public benchmarks (NSI, BLS, SIA). Everything else is my own assumption. Both are listed, with sources, in ASSUMPTIONS.md. Nothing here describes a real organization, and no result from this data says anything about real health systems.

AI-assisted code

The Python in this repo (`generate.py` , `validate.py` , `config.py`) was written with Claude, an AI assistant. I specified the data model, the assumptions, the planted effects and issues, and the checks. Claude wrote the code to my specification. I reviewed the output through the validation steps below. I don't present this as hand-written engineering work. The skill on display is the data design and the validation.

Run it

```
pip install -r ../requirements.txt
python generate.py      # writes data/ and ground_truth/   (about 10 seconds)
python validate.py      # writes validation_report.md; exits 1 if an integrity
```

Seed `20261108` is fixed in `config.py` . Same seed and same package versions give identical files. To test a different assumption, change `config.py` , not the generator.

What it supports

Case study	Question	Main tables
A. What does "complete" mean?	How many people are actually compliant, and why does the number change with the definition?	`enrollments`, `attempts`, `exemptions`, `certifications`, `lms_users`, `courses`, `employees`
D. Governed reporting	Can the monthly executive scorecard be traced, reconciled and access-controlled?	`report_published_values`, `report_lineage`, `report_catalog`, `hr_feed_runs`, `bi_users`, `bi_access_roles`
B. Does training change outcomes?	Is on-time onboarding or residency attendance linked to first-year retention?	`employees`, `enrollments` (ON-02, NR-01), `residency_attendance`, `employee_unit_history`
C. Agency vs. staff nurses	What does the agency mix cost, and where would conversion pay off?	`unit_staffing_daily`, `agency_contracts`, `proficiency`, `cost_assumptions`, `employees`

It also feeds library items LIB-03 (dashboard mockup), LIB-24 (completion notebook) and LIB-27 (data-quality script).

Folder layout

```
config.py              all assumptions (the only file to edit)
(LICENSE and LICENSE-DATA are at the repo root)
generate.py            the generator
validate.py            integrity, calibration and recovery checks
ASSUMPTIONS.md         every assumption and distribution in plain language
DATA_DICTIONARY.md     every table and column
data/                  25 CSV tables + manifest.json (row counts and seed)
ground_truth/          planted_effects.csv, planted_issues.csv (the answer key)
validation_report.md   output of validate.py
(CHANGELOG.md and SANITIZATION_LOG.md live at the repo root)
```

Ground truth

The generator plants known effects (for example, on-time onboarding lowers first-year exit risk by a hazard ratio of 0.70) and known defects (duplicate accounts, a stale month in the published scorecard). They're listed in `ground_truth/`. A case study should do its analysis first and check the answer key afterward. The point is to show whether the method finds what's there, and how far a naive method misses.

Validation

`validate.py` runs three kinds of checks:

1. **Integrity.** Primary keys are unique, every foreign key resolves, terminations follow hires, and staffing hours add up to required hours. All pass on the current seed.
2. **Calibration.** Output compared with the inputs: headcount stays between 3,900 and 4,050 and nursing at 49%; realized turnover, agency share and raw completion rates are reported next to the assumptions.
3. **Recovery.** Can the planted items be found again? A month-by-month reconciliation of the published scorecard against the raw tables flags exactly the three planted months and no others. The onboarding effect shows up in a naive comparison (ratio 0.74 against a true 0.70), but the within-tier ratios swing from 0.60 to 0.95, which is why case study B has to adjust for unit before claiming anything.

One recovery check is too thin to trust, and I'm leaving it that way. About 130 residents have a full year of follow-up, with about two dozen first-year exits. Early attendance points the right way, but the estimate is too noisy to report as a finding. That's realistic for a system this size, and case study B uses it to show a power limit rather than hiding it.

Limits

- Benchmarks are national averages from hospital surveys; clinic and home-health rates are extrapolated. Learning-record behavior has no public benchmark.
- Only RN hours are simulated by unit and day.
- No patient or clinical data, no demographics, no compensation by person.
- Names are random combinations from short lists of common names, so different people share names on purpose.
- Training names are generic, and nothing here implies clinical or regulatory expertise.

License and disclaimer

Code: MIT (`../LICENSE`). Synthetic data in `data/` and `ground_truth/` : CC0 1.0 (`../LICENSE-DATA`). The public benchmarks in `config.py` are cited facts, credited to their sources in ASSUMPTIONS.md.

Matthew McKay, Ph.D.

Personal project. All data is synthetic or public, and the company is fictional. Views are my own and not affiliated with or endorsed by any current or former employer.